

CS305 SDN Project Report (中文版)

Tong Wu (12412460) Zhikai Wang (12410615)
Yunzang Li (12410922)

2026 年 6 月 4 日

1 项目简介

本项目基于 os-ken 与 Mininet 实现了一个集中式 SDN 控制器。网络被建模为单子网的二层环境，包含主机与 OpenFlow 交换机。控制器支持三项必做功能：

- 基于 DHCP 的动态地址分配；
- 基于当前拓扑的最短路径转发；
- 基于交换机流表的防火墙过滤。

在必做部分之外，我们还实现了多个 bonus，包括 DNS 支持、DHCP 租约增强、TCP 拥塞控制实验以及 bufferbloat 实验。

2 系统架构

整个系统以集中式控制器为核心。Mininet 负责创建虚拟主机、交换机与链路；os-ken 控制器维护全局拓扑视图，并将 OpenFlow 规则下发到交换机中。主机负责产生业务流量，而控制器负责决定网络应当如何响应。

从功能上看，系统可以分为四层：

- 拓扑与转发层：维护交换机、链路和主机状态；计算最短路径；安装转发表。
- **DHCP** 层：处理 DHCP 报文，管理地址分配、pending offer、正式租约与续租。
- 防火墙层：从 JSON 规则文件中解析 deny 规则，并下发高优先级丢弃流表。
- **Bonus DNS** 层：按照静态 DNS 记录表响应域名查询。

2.1 核心文件

- `controller.py`：控制器主逻辑，处理交换机、主机、链路、端口与数据包事件，并协调 DHCP、DNS、ARP、最短路和防火墙规则安装。

- `dhcp.py` : DHCP 服务器实现 , 包含地址分配、租约管理、超时回收与续租逻辑。
- `firewall.py` : 防火墙规则解析与流表安装逻辑。
- `dns_server.py` : 控制器侧 bonus DNS 实现。
- `ofctl_utilis.py` : OpenFlow 辅助工具 , 负责流表安装和 PacketOut 发送。
- `tests/` : Mininet 测试脚本 , 覆盖必做和 bonus 部分。

2.2 控制逻辑

控制器维护以下共享状态 :

- 在线 datapath 与 OpenFlow 控制抽象 ;
- 学习到的主机位置以及 IP 到 MAC 的映射 ;
- 交换机邻接关系 ;
- 交换机间端口映射 ;
- 用于拓扑刷新和故障处理的 down-port 集合。

在此基础上 , 控制器根据主机位置与交换机拓扑计算最短路径 , 并按照目的 MAC 下发转发规则。

3 实现细节

3.1 DHCP

DHCP 模块实现了标准的 DISCOVER/OFFER/REQUEST/ACK 流程。测试脚本中的主机初始并不带有效 IP , 因此地址分配必须由控制器侧完成。

`dhcp.py` 中维护了如下状态 :

- pending offers ;
- 已确认租约 ;
- 租约过期时间戳 ;
- 被客户端拒绝、不得再次分配的地址。

在基础流程之外 , 我们还对 DHCP 做了增强 :

- 支持自定义地址池范围与子网掩码 ;
- 支持 OFFER 超时回收 ;
- 支持租约过期回收与续租 ;
- 支持并发请求下的无冲突地址分配。

3.2 拓扑学习与最短路径交换

交换部分主要实现在 `controller.py` 中。控制器会：

- 通过 gratuitous ARP 与普通 ARP 学习主机位置；
- 通过 LLDP 与 topology API 学习交换机间链路；
- 在交换机、主机、链路、端口事件发生时更新内部拓扑状态；
- 在交换机图上使用 BFS 计算最短路径；
- 基于目的 MAC 向交换机安装转发规则。

该设计既支持基础三角拓扑，也支持带环和动态变化的复杂拓扑。

3.3 防火墙

防火墙模块从 `firewall_rule.json` 中读取策略。每条规则可匹配源 IP、目的 IP、传输层协议、源端口和目的端口。模块将 deny 规则转换为高优先级 OpenFlow 丢弃流表。

这些规则会通过 `Firewall.install_rules` 安装到所有交换机上。由于防火墙规则优先级高于普通转发规则，因此一旦报文匹配 deny 规则，将直接被拦截。

3.4 Bonus Components

DNS. DNS bonus 在控制器侧实现了一个轻量级 DNS responder。控制器会识别 DNS 请求，并按照 `dns_records.json` 中的静态记录构造响应。

DHCP Lease Management and RFC Optimization. DHCP bonus 在基础地址分配之上增加了更完整的租约生命周期管理和 RFC 2131 兼容性，包括租约过期回收、OFFER 超时清理、续租、原子 reservation、防止重复分配以及 DECLINE 冲突处理。

TCP Congestion. TCP congestion bonus 是 Mininet 实验，而不是控制器侧功能。它在相同瓶颈链路上对比 Reno 与 CUBIC 的吞吐量与公平性。

Bufferbloat. Bufferbloat bonus 也是 Mininet 实验，用于展示不同队列大小下吞吐量与排队时延之间的权衡。

4 团队分工

成员	主要职责	关键文件
Tong Wu (12412460)	DHCP 基础流程、租约管理、避免重复分配、DHCP option 处理，以及 DHCP 相关测试	dhcp.py, tests/dhcp_test/
Zhikai Wang (12410615)	拓扑事件处理、ARP 处理、最短路径计算、转发规则安装、动态拓扑更新与复杂拓扑设计	controller.py, ofctl_utilis.py, tests/switching_test/
Yunzang Li (12410922)	防火墙规则加载与安装、bonus 实验、报告与图片整理、演示脚本支持	firewall.py, firewall_rule.json, tests/firewall_test/ report/

5 测试方法

我们使用基于 Mininet 的脚本验证基础功能与复杂动态场景。必做测试主要分为三类。

5.1 DHCP Tests

- 默认配置下的基础 DHCP 分配；
- 修改 start_ip、end_ip 和 netmask 后的自定义 DHCP 配置；
- 地址池耗尽场景，主机数量超过地址池容量。

```

(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
Sending DHCP request dhclient -v h1-eth0
Sending DHCP request dhclient -v h2-eth0
*** Starting CLI:
mininet> h1 ifconfig h1-eth0
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.2 netmask 255.255.255.0 broadcast 192.168.1.255
    ether 00:00:00:00:00:01 txqueuelen 1000 (Ethernet)
    RX packets 5 bytes 1068 (1.0 KB)
    RX errors 0 dropped 1 overruns 0 frame 0
    TX packets 6 bytes 1268 (1.2 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

mininet> h2 ifconfig h2-eth0
h2-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.3 netmask 255.255.255.0 broadcast 192.168.1.255
    ether 00:00:00:00:00:02 txqueuelen 1000 (Ethernet)
    RX packets 5 bytes 1068 (1.0 KB)
    RX errors 0 dropped 1 overruns 0 frame 0
    TX packets 6 bytes 1268 (1.2 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

mininet> h1 ping -c 2 192.168.1.3
PING 192.168.1.3 (192.168.1.3) 56(84) bytes of data.
64 bytes from 192.168.1.3: icmp_seq=1 ttl=64 time=4.06 ms
64 bytes from 192.168.1.3: icmp_seq=2 ttl=64 time=0.077 ms

--- 192.168.1.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1010ms
rtt min/avg/max/mdev = 0.077/2.070/4.064/1.993 ms
mininet>

```

图 1: 基础 DHCP 测试结果，展示成功分配地址并完成连通性验证。

```

*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
h1: Sending DHCP request on eth0
h2: Sending DHCP request on eth0
h3: Sending DHCP request on eth0

===== Assigned Addresses =====
h1 -> 10.10.0.10
h2 -> 10.10.0.11
h3 -> 10.10.0.12

===== Connectivity Check h1 -> h2 =====
PING 10.10.0.11 (10.10.0.11) 56(84) bytes of data.

--- 10.10.0.11 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1028ms

*** Stopping 1 controllers
c0
*** Stopping 3 links
...
*** Stopping 1 switches
s1
*** Stopping 3 hosts
h1 h2 h3
*** Done

[PASS] DHCP custom configuration test passed

```

图 2: 自定义地址池与子网掩码下的 DHCP 测试结果。

```

s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1) (h5, s1)
*** Configuring hosts
h1 h2 h3 h4 h5
*** Starting controller
c0
*** Starting 1 switches
s1 ...
h1: Sending DHCP request on eth0
h2: Sending DHCP request on eth0
h3: Sending DHCP request on eth0
h4: Sending DHCP request on eth0
h5: Sending DHCP request on eth0

==== Allocation Results ====
h1 -> 10.10.0.10
h2 -> None
h3 -> 10.10.0.12
h4 -> 10.10.0.11
h5 -> None

Expected: first 3 hosts receive IPs, remaining hosts do not.
Allocated hosts: h1, h3, h4
Unallocated hosts: h2, h5
*** Stopping 1 controllers
c0
*** Stopping 5 links
.....
*** Stopping 1 switches
s1
*** Stopping 5 hosts
h1 h2 h3 h4 h5
*** Done

[PASS] DHCP pool exhaustion test passed

```

图 3: 地址池耗尽测试结果，只有前几个主机获得地址。

5.2 Shortest-Path Switching Tests

- 基础三角拓扑下的全连通验证；
- 超过 6 台主机、6 台交换机、10 条边的复杂拓扑；
- 链路 down/up、交换机 stop/start、端口 disable/enable 等动态变化。

```

bin/python tests/switching_test/test_network.py
*** Creating network
*** Adding controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1 s2 s3
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (s1, s2) (s2, s3) (s3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 3 switches
s1 s2 s3 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)

```

图 4: 基础三角拓扑中的交换与连通性验证。

5.3 Firewall Tests

- 单交换场景下的基础防火墙策略验证；
- 复用复杂交换拓扑的多交换机防火墙场景；
- 验证指定通信被阻断，而无关流量仍可正常通过。

```
bin/python tests/firewall_test/test_network.py
*** Creating network
*** Adding controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
PING 192.168.117.3 (192.168.117.3) 56(84) bytes of data.

--- 192.168.117.3 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1040ms

PING 192.168.117.4 (192.168.117.4) 56(84) bytes of data.
64 bytes from 192.168.117.4: icmp_seq=1 ttl=64 time=3.34 ms
64 bytes from 192.168.117.4: icmp_seq=2 ttl=64 time=0.107 ms

--- 192.168.117.4 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1026ms
rtt min/avg/max/mdev = 0.107/1.725/3.344/1.618 ms

curl: (28) Failed to connect to 192.168.117.3 port 80 after 2000 ms: Timeout was reached
HTTP_CODE=000

HTTP_CODE=200
```

图 5: 基础防火墙测试结果。

```
curl: (28) Failed to connect to 192.168.117.3 port 80 after 2010 ms: Timeout was reached
HTTP_CODE=000
EXIT=28

[PASS] h1 -> h2 TCP/8080 is allowed
HTTP_CODE=200
EXIT=0

[PASS] h3 -> h4 ICMP remains reachable
PING 192.168.117.5 (192.168.117.5) 56(84) bytes of data.
64 bytes from 192.168.117.5: icmp_seq=1 ttl=64 time=0.611 ms

--- 192.168.117.5 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.611/0.611/0.611/0.000 ms

[PASS] h6 -> h4 TCP/8081 remains reachable
HTTP_CODE=200
EXIT=0

[PASS] h7 -> h5 ICMP remains reachable
PING 192.168.117.6 (192.168.117.6) 56(84) bytes of data.
64 bytes from 192.168.117.6: icmp_seq=1 ttl=64 time=4.74 ms

--- 192.168.117.6 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 4.736/4.736/4.736/0.000 ms

[PASS] Reverse h2 -> h1 TCP/8081 is allowed
HTTP_CODE=200
EXIT=0

===== Summary: 7/7 checks passed =====

===== Firewall Flow Entries =====
```

图 6: 复杂拓扑下的防火墙测试结果。

6 复杂测试案例

项目要求设计一个超过 6 台主机、6 台交换机、10 条边的复杂拓扑。我们构建了符合要求的拓扑，并将其同时用于交换和防火墙部分的演示。

6.1 拓扑概要

- Hosts: 7
- Switches: 7
- Inter-switch edges: 12

完整的 Mermaid 拓扑图与主机间最短路径表整理在 docs/complex-case-topology.md 中。

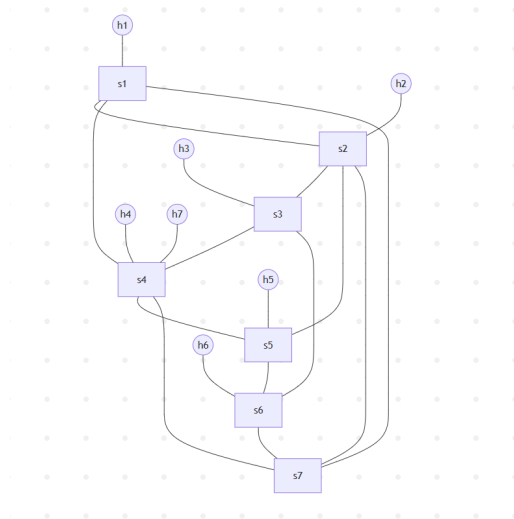


图 7: 扩展演示中使用的复杂拓扑。

6.2 动态操作

该复杂案例支持以下 Mininet CLI 操作：

- link down/up ;
- switch stop/start ;
- 通过 `ovs-ofctl mod-port` 进行 port down/up。

这些操作用于展示控制器在拓扑变化时重新计算路径并刷新流表的能力。

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7
h2 -> h1 h3 h4 h5 h6 h7
h3 -> h1 h2 h4 h5 h6 h7
h4 -> h1 h2 h3 h5 h6 h7
h5 -> h1 h2 h3 h4 h6 h7
h6 -> h1 h2 h3 h4 h5 h7
h7 -> h1 h2 h3 h4 h5 h6
*** Results: 0% dropped (42/42 received)
mininet> link s2 s5 down
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7
h2 -> h1 h3 h4 h5 h6 h7
h3 -> h1 h2 h4 h5 h6 h7
h4 -> h1 h2 h3 h5 h6 h7
h5 -> h1 h2 h3 h4 h6 h7
h6 -> h1 h2 h3 h4 h5 h7
h7 -> h1 h2 h3 h4 h5 h6
*** Results: 0% dropped (42/42 received)
mininet> link s2 s5 up
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7
h2 -> h1 h3 h4 h5 h6 h7
h3 -> h1 h2 h4 h5 h6 h7
h4 -> h1 h2 h3 h5 h6 h7
h5 -> h1 h2 h3 h4 h6 h7
h6 -> h1 h2 h3 h4 h5 h7
h7 -> h1 h2 h3 h4 h5 h6
*** Results: 0% dropped (42/42 received)
```

图 8: 复杂拓扑动态变化过程中的连通性验证结果。

7 Bonus 结果与分析

7.1 DHCP Bonus

DHCP bonus 的重点，是将基础地址分配器扩展为一个更完整的 DHCP 服务。服务器侧维护四类核心状态：pending offers、confirmed leases、lease expiration timestamps，以及被客户端拒绝、不得再次分配的地址。基于这几类状态，系统能够完成租约回收、续租与冲突处理。

7.1.1 DHCP 状态机

本实现遵循 RFC 2131 的客户端生命周期，而不是把 DHCP 简化成一次性的 DORA 交换。服务器能够区分正常的 DISCOVER -> OFFER -> REQUEST -> ACK 路径，以及 INIT-REBOOT、RENEWING、REBINDING 等不同类型的 REQUEST；同时，也支持显式的 RELEASE 与 DECLINE。因此，它实现的是租约管理逻辑，而不是一次性的地址发放。

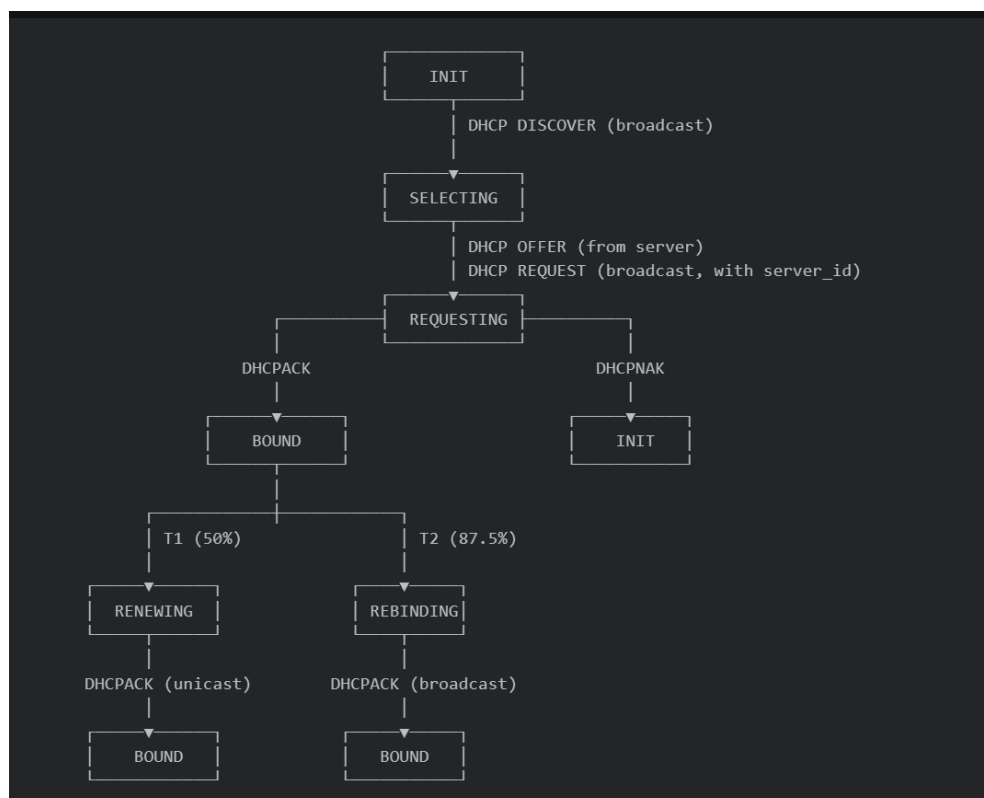


图 9: RFC 2131 中客户端 DHCP 状态迁移图，对应本实现中的租约管理逻辑。

该状态机在 `dhcp.py` 中有直接对应。REQUEST 处理逻辑不会把所有 REQUEST 一概而论，而是根据 `ciaddr`、Option 50 和 Option 54 识别语义，再执行不同的 RFC 处理路径。

```

if ciaddr == '0.0.0.0' and requested_ip and server_id_opt:
    # SELECTING
    ...

if ciaddr == '0.0.0.0' and requested_ip and not server_id_opt:
    # INIT-REBOOT
    ...

if ciaddr and ciaddr != '0.0.0.0':
    # RENEWING / REBINDING
    ...
  
```

7.1.2 Lease Expiration

租约过期决定了地址何时能够在无人干预的情况下返回地址池。服务器为每个正式租约维护一个过期时间戳，并在当前时间超过该时间点后回收该租约。

```

@classmethod
def _is_expired(cls, ip: str) -> bool:
  
```

```

    expiry = cls.lease_expiry.get(ip)
    return expiry is not None and time.time() > expiry

@classmethod
def _reclaim_expired(cls):
    now = time.time()

    expired_macs = [
        mac for mac, ip in cls.ip_pool.items()
        if cls._is_expired(ip)
    ]
    for mac in expired_macs:
        ip = cls.ip_pool.pop(mac)
        cls.lease_expiry.pop(ip, None)

```

在报告展示中，我们将 □ 自然过期测试 □ 作为这一部分的主示例。租期到达后，服务器侧旧租约被回收，客户端清空旧状态后重新发起请求，并再次获得可用租约。这个过程直接验证了过期与复用机制。

```

Step 1: Request initial DHCP lease
h1: Sending DHCP request on eth0
Initial IP: 192.168.1.2
Lease start time: 00:39:28
Lease will expire at: 00:39:58

Step 2: Stopping dhclient to prevent auto-renewal
This is critical - dhclient would auto-renew at T1 (50%% of lease time)
dhclient stopped

Step 3: Waiting for lease to expire (30 seconds)...
Remaining: 5 seconds
Lease has expired on server side!

Step 4: Verify lease expiry by requesting new DHCP lease
Note: Client IP may still be in kernel, but server lease has expired
Clearing old IP configuration...
Current IP after flush: None

Step 5: Requesting new DHCP lease...
h1: Sending DHCP request on eth0
New IP after expiry: 192.168.1.2
[PASSED]: Client successfully obtained new lease after expiry

```

图 10: 租约过期测试：原租约在服务器侧自然失效，客户端清空旧配置后再次成功获取新租约。

7.1.3 Duplicate-IP Prevention

为了避免两个客户端获得同一个地址，分配器采用了原子 reservation 机制。它在分配新地址前会先回收过期资源，然后同时检查正式租约表与 pending-offer 表；一旦找到可用地址，就立即写入 pending_offers。这样即使多个客户端在相近时间发送 DISCOVER，也不

会观察到同一个空闲地址。

```
@classmethod
def _allocate_ip(cls, client_mac: str) -> Optional[str]:
    cls._reclaim_expired()

    if client_mac in cls.ip_pool:
        return cls.ip_pool[client_mac]
    if client_mac in cls.pending_offers:
        return cls.pending_offers[client_mac]["ip"]

    in_use = cls._in_use_ips()
    for ip in cls._build_ip_range():
        if ip not in in_use and ip not in cls.bad_ips:
            cls.pending_offers[client_mac] = {
                "ip": ip,
                "timestamp": time.time(),
            }
            return ip
    return None
```

并发分配测试在开发过程中用于验证实现，但截图本身并不能像分配逻辑与日志那样直接说明“无重复 IP”这一性质。因此最终报告保留机制说明，而不再放入两张并发测试截图。

7.1.4 OFFER Timeout Reclaim

服务器也会清理过期的 pending_offers。如果客户端收到了 OFFER，但没有继续发送 REQUEST 确认，那么这个 reservation 会在 offer_timeout 秒后被释放，地址随之重新进入可分配状态。

```
expired_offer_macs = [
    mac for mac, info in cls.pending_offers.items()
    if now - info["timestamp"] > cls.offer_timeout
]
for mac in expired_offer_macs:
    info = cls.pending_offers.pop(mac)
```

```

Step 1: h1 sends DISCOVER (only DISCOVER, no REQUEST)
h1: Sending DHCP DISCOVER on eth0
DISCOVER sent at: 07:52:32
OFFER will expire at: 07:52:42

Step 2: Waiting 13 seconds for OFFER to timeout...
Remaining: 1 seconds
OFFER should have expired now!

Step 3: h1 re-sends DISCOVER
h1: Sending DHCP DISCOVER on eth0
h1 re-sent DISCOVER, pending offer should have been reclaimed

Step 4: h1 completes DORA
h1: Sending DHCP request on eth0
h1 IP: 192.168.1.2
[PASSED]: h1 obtained IP after pending OFFER expired

```

图 11: DHCP OFFER 超时回收测试结果。

7.1.5 Lease Renewal

对于 RENEWING 和 REBINDING，客户端已经持有地址，因此服务器需要验证 ciaddr 是否确实仍属于当前 MAC，再延长租约时间。这样可以保证续租操作仍然绑定到正确的租约记录。

```

Step 1: Get initial DHCP lease
h1: Sending DHCP request on eth0
Initial IP: 192.168.1.2
Original lease expiry time: 07:58:40

Step 2: Waiting 18 seconds for T1 (auto-renewal time)...
IP at T1 time: 192.168.1.2

Step 3: Wait until original expiry time...
Waiting 11 seconds to reach original expiry...

Step 4: Verify IP is still valid at original expiry time
IP at original expiry: 192.168.1.2
[PASSED]: IP still valid at original expiry (renewal successful)

```

图 12: 租约续租测试：客户端在租约失效前完成续租，并保持原有地址。

```

if ciaddr and ciaddr != '0.0.0.0':
    existing_ip = cls.ip_pool.get(client_mac)
    if existing_ip != ciaddr:
        return cls._build_nak_pkt(client_mac, xid)
    if cls._is_expired(ciaddr):
        cls.ip_pool.pop(client_mac, None)
        cls.lease_expiry.pop(ciaddr, None)
        return cls._build_nak_pkt(client_mac, xid)

    cls.lease_expiry[ciaddr] = time.time() + cls.lease_time

```

```

    return cls._build_reply_pkt(
        client_mac, xid, ciaddr, dhcp.DHCP_ACK,
        unicast_ip=ciaddr
    )

```

7.1.6 DHCPNAK Handling

对于非法或不一致的请求，服务器会构造 RFC 兼容的 DHCPNAK。特别地，返回报文中的 yiaddr 为 0.0.0.0，并且通过广播发送，因为客户端在该时刻可能并不持有可用地址。

```

@classmethod
def _build_nak_pkt(cls, client_mac: str, xid: int) -> packet.Packet:
    dhcp_pkt = dhcp.dhcp(
        op=dhcp.DHCP_BOOT_REPLY,
        chaddr=client_mac,
        siaddr=cls._gateway_ip,
        yiaddr='0.0.0.0',
        xid=xid,
        boot_file='',
        options=options,
    )

```

7.1.7 DECLINE and RELEASE

实现中还支持客户端主动释放租约以及上报地址冲突。RELEASE 会立即把地址归还给地址池；DECLINE 会将该地址从当前状态表中移除，并加入 bad_ips，从而避免冲突地址再次被分配。

```

@classmethod
def _handle_release(cls, pkt):
    ip = cls.ip_pool.pop(client_mac, None)
    if ip:
        cls.lease_expiry.pop(ip, None)
        cls.pending_offers.pop(client_mac, None)

@classmethod
def _handle_decline(cls, pkt):
    ...
    cls.bad_ips.add(declined_ip)

```

7.1.8 Broadcast and Unicast Reply Selection

回复路径明确区分了“初始化阶段”和“续租阶段”。对于尚未获得地址的客户端，OFFER 和初始 ACK 采用广播回复；而对于已经持有地址的续租场景，则可以单播 ACK 到客户

端现有地址。

```
use_broadcast = unicast_ip is None
dst_mac = 'ff:ff:ff:ff:ff:ff' if use_broadcast else client_mac
dst_ip = '255.255.255.255' if use_broadcast else unicast_ip
```

7.2 DNS Bonus

DNS bonus 展示了控制器不仅能做转发和 DHCP，也能充当一个简化版 DNS responder。

该实现完全位于控制器侧。启动时会从静态 JSON 文件读取 DNS server 的虚拟 IP、虚拟 MAC 以及域名到 IP 的映射表。当交换机加入时，控制器会安装一条高优先级的 PacketIn 规则，匹配 IPv4 UDP 53 端口，从而把 DNS 请求送到 controller，而不是走普通的最短路径转发。同时，控制器还会对虚拟 DNS IP 192.168.1.1 的 ARP 请求进行应答，保证主机能够向这个虚拟 DNS server 发出 UDP 报文。

DNS 响应路径实现在 dns_server.py 中。控制器从 UDP payload 中取出 DNS query，解析 question，查静态记录表，然后手工构造完整的 DNS response 以太网帧，并通过 PacketOut 返回给请求主机。

```
def install_packetin_flow(self, datapath, ofctl, ether_types, inet, vlan_id):
    actions = [
        datapath.ofproto_parser.OFPActionOutput(
            datapath.ofproto.OFPP_CONTROLLER,
            0xFFFF,
        )
    ]
    ofctl.set_flow(
        cookie=self.COOKIE,
        priority=self.PACKETIN_PRIORITY,
        dl_type=ether_types.ETH_TYPE_IP,
        dl_vlan=vlan_id,
        nw_proto=inet.IPPROTO_UDP,
        tp_dst=self.DNS_PORT,
        actions=actions,
    )
```

```
def build_response(self, query):
    query_id, flags, qdcount, _, _, _ = struct.unpack("!HHHHHH", query[:12])
    question_end, qname = self._read_qname(query, 12)
    question = query[12:question_end + 4]
    qtype, qclass = struct.unpack("!HH", query[question_end:question_end + 4])
```



```

answer_ip = self.records.get(self._normalize_name(qname))

answer = b""
rcode = 0
if qtype == 1 and qclass == 1 and answer_ip:
    answer = self._build_a_answer(answer_ip)
elif qtype == 1 and qclass == 1:
    rcode = 3

response_flags = 0x8000 | 0x0400 | (flags & 0x0100) | rcode
header = struct.pack("!HHHHHH", query_id, response_flags, 1, 1 if
answer else 0, 0, 0)
return header + question + answer

```

测试拓扑为简单的 h1-s1-h2 场景。测试脚本验证三件事：web.cs305.local 能否解析为 192.168.1.3，h1.cs305.local 能否解析为 192.168.1.2，以及不存在的域名能否返回 NXDOMAIN。此外，脚本还调用 pingAll() 验证 DNS 扩展没有破坏普通转发，并打印交换机流表，确认 UDP/53 的 PacketIn 规则确实存在。

```

h2 -> h1
*** Results: 0% dropped (2/2 received)

===== DNS bonus test results =====

[PASS] web.cs305.local resolves to 192.168.1.3
DNS_FLAGS=0x8500 ANSWERS=1 FROM=192.168.1.1
DNS_TEST_PASS
EXIT=0

[PASS] h1.cs305.local resolves to 192.168.1.2
DNS_FLAGS=0x8500 ANSWERS=1 FROM=192.168.1.1
DNS_TEST_PASS
EXIT=0

[PASS] missing.cs305.local returns NXDOMAIN
DNS_FLAGS=0x8503 ANSWERS=0 FROM=192.168.1.1
DNS_TEST_PASS
EXIT=0

[PASS] normal shortest-path forwarding still works
packet loss=0.0%

===== DNS PacketIn flow on s1 =====
cookie=0x305d, duration=8.339s, table=0, n_packets=1, n_bytes=75, idle_age=0, priority=2000,udp,tp_dst=53 actions=CONTROLLER:65535
*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done

===== Summary: 4/4 checks passed =====
(cs305) w1369@LAPTOP-UPKABV41:~/computer_network/make_proj/CS305-26Spring-Project$
[work] 0:[tmux] 1:node 2:python3.8- 3:bash* 4:bash "LAPTOP-UPKABV41" 10:59 04-Jun-26

```

图 13: DNS bonus 测试结果。

7.3 TCP Congestion Bonus

TCP congestion bonus 用于比较不同拥塞控制算法在相同瓶颈链路上的行为差异。实验结果表明，不同算法会带来不同的吞吐表现，而在双流共享瓶颈时，fairness 通常仍然接近 1。

这一 bonus 是 Mininet 实验，而不是控制器内功能。拓扑中包含四台主机和两台交换机，s1 与 s2 之间有一条唯一瓶颈链路。接入链路配置为 100 Mbps，1 ms，瓶颈链路配置为 10 Mbps，30 ms，0.5% loss，queue size 100。其中 0.5% 的随机丢包由 TCLink 直接设置，用于构造一个可控的拥塞信号环境，以观察 Reno 和 CUBIC 在丢包后的恢复差异。

脚本包含两类实验。第一类是单流实验：h1 -> h2 分别在 Reno 和 CUBIC 下各运行一次。第二类是公平性实验：h1 -> h2 使用 Reno，h3 -> h4 使用 CUBIC，同时共享同一条瓶颈链路。吞吐量通过 iperf 输出解析，公平性通过 Jain's fairness index 计算。

```
def set_tcp_algorithm(host, algorithm):
    host.cmd("modprobe tcp_%s 2>/dev/null || true" % algorithm)
    output = host.cmd(
        "sysctl -w net.ipv4.tcp_congestion_control=%s 2>&1" % algorithm
    )
    current = host.cmd("sysctl -n net.ipv4.tcp_congestion_control 2>/dev/
null").strip()
    if current != algorithm:
        return False
    return True
```

```
def jain_fairness(x1, x2):
    if x1 is None or x2 is None:
        return None
    denominator = 2 * (x1 ** 2 + x2 ** 2)
    if denominator == 0:
        return None
    return (x1 + x2) ** 2 / denominator
```

从定性预期来看，CUBIC 在单流场景下通常会比 Reno 有更高的平均吞吐量，因为它在丢包后的窗口恢复更积极；与此同时，公平性实验的结果一般仍会接近 1，说明两条流即使采用不同算法，也能在瓶颈链路上较为均衡地共享带宽。我们多次实验的结果也符合这一趋势：CUBIC 的平均单流吞吐量更高，而整体 fairness 保持较高水平。

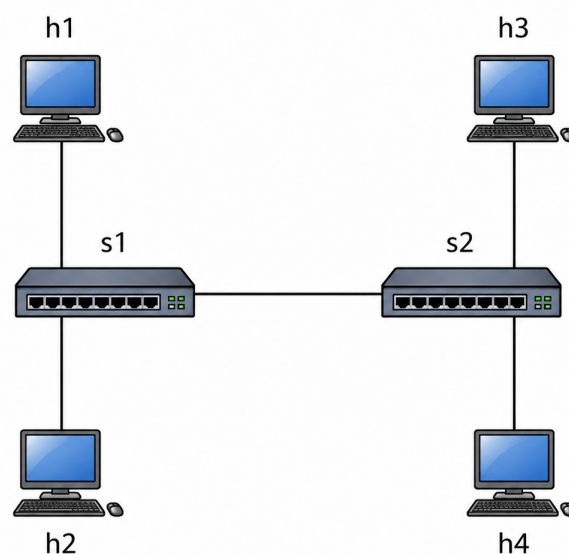


图 14: TCP congestion 与 bufferbloat bonus 共用的 Mininet 实验拓扑。

```

===== Result Summary =====

Single flow throughput:
algorithm    loss(%) throughput(Mbps)
reno         0.5      4.05
cubic        0.5      4.43

Fairness test:
reno(Mbps)   cubic(Mbps)   fairness
3.9          2.95       0.9811291165708309

Conclusion data:
single_reno(Mbps)  single_cubic(Mbps)  fair_reno(Mbps)  fair_cubic(Mbps)  fairness
4.05              4.43              3.9              2.95              0.9811291165708309

```

图 15: Reno、CUBIC 以及 fairness 测试的 TCP congestion 实验结果。

7.4 Bufferbloat Bonus

Bufferbloat 实验展示了经典的吞吐量与时延权衡：当队列非常大时，丢包率可以下降，TCP 吞吐量也可能上升，但在有大流量时，交互流量的 RTT 会显著膨胀。

该实验沿用两台交换机的基本结构，但修改了瓶颈链路配置。接入链路保持为 100 Mbps，1 ms，而 s1-s2 的瓶颈链路设置为 1 Mbps，20 ms。实验中唯一变化的参数是瓶颈队列大小：一组使用 20 packets，另一组使用 1000 packets。

实验同时运行两类流量：h1 -> h3 使用 iperf 产生长时间 TCP 大流量，h2 -> h4 使用 ping 作为交互式小包探测流。脚本先测量空闲状态下的 RTT baseline，然后启动后台 ping，再启动前台 TCP 大流，最后分别解析 iperf 输出和 ping 统计结果。

```

def parse_ping(output):
    loss_match = re.search(r"(\d+(?:\.\d+)?)% packet loss", output)
    rtt_match = re.search(
        r"(?:rtt|round-trip) min/avg/max/(?:mdev|stddev) = "
        r"[\d.]+/([\d.]+)/[\d.]+/[\d.]+",
        output,
    )
    loss = float(loss_match.group(1)) if loss_match else None
    avg_rtt = float(rtt_match.group(1)) if rtt_match else None
    return loss, avg_rtt

```

```

idle_ping = ping(h2, h4.IP())
idle_loss, idle_rtt = parse_ping(idle_ping)
h3.cmd("iperf -s -p 5001 >/tmp/bonus5_iperf_server.log 2>&1 &")
h2.cmd("ping -c 30 -i 0.5 -W 2 %s >/tmp/bonus5_ping.log 2>&1 &" % h4.IP())
iperf_output = h1.cmd("iperf -c %s -p 5001 -t %s" % (h3.IP(), duration))
busy_ping = h2.cmd("cat /tmp/bonus5_ping.log")
busy_loss, busy_rtt = parse_ping(busy_ping)
throughput = parse_iperf(iperf_output)

```

这一实验的预期结果，就是典型的 bufferbloat 现象：大队列会减少早期丢包，也可能提升 TCP 吞吐量，因为更多数据包被缓存而不是直接丢弃；但与此同时，小包交互流量必须排

在大流量后面等待，因此 busy RTT 会非常明显地上升。我们的实验结果也正是如此：大队列下吞吐更高、丢包更低，但拥塞时的 RTT 显著增加，清楚展示了 bufferbloat 的核心问题。

```

*** Starting controller

*** Starting 2 switches
s1 s2 ...(100.00Mbit 1ms delay) *** Error: Warning: sch_htb: quantum of class 50001
r2q change.
(100.00Mbit 1ms delay) *** Error: Warning: sch_htb: quantum of class 50001
e.
(1.00Mbit 20ms delay) (100.00Mbit 1ms delay) *** Error: Warning: sch_htb: q
ig. Consider r2q change.
(100.00Mbit 1ms delay) *** Error: Warning: sch_htb: quantum of class 50001
e.
(1.00Mbit 20ms delay)

===== Queue size: 1000 packets =====
*** Stopping 0 controllers

*** Stopping 5 links
.....
*** Stopping 2 switches
s1 s2
*** Stopping 4 hosts
h1 h2 h3 h4
*** Done

===== Summary =====
queue   throughput(Mbps)   idle_rtt(ms)   busy_rtt(ms)   loss(%)
20      0.954               50.961         327.488        13.3333
1000    1.3                 51.227         1820.666       0.0

```

图 16: 不同瓶颈队列大小下的 bufferbloat 实验结果。

8 总结

本项目实现了一个可运行的 SDN 控制器，将 DHCP、最短路径交换和防火墙功能集成在同一个 os-ken 应用中。整个实现由基于 Mininet 的测试支撑，并通过复杂拓扑与 bonus 实验进一步展示了系统在功能扩展和网络实验方面的能力。

最终报告应结合仓库中的测试脚本与演示文档一同阅读，尤其是复杂拓扑说明和现场答辩指南。